Task 1:

The provided `shapes.py` file contains three classes: `Shape`, `Polygon`, and `RightTriangle`.

Add a `Rectangle` class that inherits `Polygon` (why? For each potential superclass, ask yourself *Is a `Rectangle` an X?*). Then add the following methods:

- A constructor, which takes the `width` and `height` of the rectangle
- A `getArea` method, which takes no parameters
- A `display` method, which again takes no parameters. `display` should first call display from Rectangle's superclass, and then print the `Rectangle`'s area.

Next, add a `Circle` class. Answer the question above to figure out which class `Circle` ought to inherit from. `Circle` needs the following methods:

- A constructor, which takes the `Circle`'s `radius`
- A `getArea` method, which takes no parameters
- A `getCircumference` method, which likewise takes no parameters
- A `display` method, also with no parameters. As above, first call `display` from the superclass; then print out the area and circumference.

Lastly, add a `Square` class. Should it inherit from the `Shape`, `Polygon` or `Rectangle` class? Do we need to implement `getArea` or `display` methods?

Test your code by adding the following lines to your `main()` function

```
C = Circle(2)
C.display()
print()

T = RightTriangle(3,4)
T.display()
print()

R = Rectangle(3,4)
R.display()
print()
```

```
S = Square(2)
S.display()
```

It should print

```
I am a Circle
My area is 12.57
My circumference is 12.57

I am a Right Triangle
I have 3 sides.
My perimeter is 12.0
My area is 6.0

I am a Rectangle
I have 4 sides.
My perimeter is 14
My area is 12

I am a Square
I have 4 sides.
My perimeter is 8
My area is 4
```

Task 2:

Create script `bank.py` and add a class called `Account` which is meant to represent a banking account

- Add a constructor which takes an `owner` and `balance`, where `owner` is a string containing the name of the account owner and `balance` is the initial amount of money in the account
- Add a `getBalance` method which returns the amount of money in the account
- Add a `getOwner` method which returns the name of the account owner
- Add a toString method which prints the `owner` and the `balance`.

```
>>> a = Account("Alex", 1000)
>>> print(a)
Alex's account contains $1000.00 dollars
```

- Add a `withdraw` method which subtracts a given amount from the `balance`.

```
>>> a = Account("Alex", 1000)
>>> print(a.getBalance())
1000
>>> a.withdraw(50)
>>> print(a.getBalance())
950
```

- Add a `deposit` method which adds an amount to the balance.

```
>>> b = Account("Beth", 10000)
>>> print(b.getBalance())
10000
>>> b.deposit(1)
>>> print(b.getBalance())
10001
```

- Add a `transferTo` method which transfers an amount to another account. Usage should be

```
>>> a = Account("Alex", 1000)
>>> b = Account("Beth", 10000)
>>> b.transferTo(a, 25)
>>> print(a.getBalance())
1025
>>> print(b.getBalance())
9975
```

- Add a `getHistory` method that returns the balance history as a list. You will need to modify the constructor, `withdraw`, `deposit` and `transferTo` methods to record the history. Note that this does **not** add any additional parameters to the constructor!

```
>>> c = Account("Carla", 200)
>>> c.withdraw(50)
>>> c.deposit(75)
>>> print(c.getHistory())
[200, 150, 225]
```

Add a class called `SavingsAccount` which inherits from your `Account` class

- Add a constructor which takes the `owner`, `balance`, and `interest` where an `interestRate` is a percent, between 0 and 1
- Add a method `getInterestRate` which returns the interest rate the account
- Add a method `addInterest` which adds interest to the balance. Usage should be

```
>>> d = SavingsAccount("Dan", 400, .01)
>>> d.addInterest()
>>> print(d)
Dan's account contains $404.00 dollars
>>> d.addInterest()
```

```
>>> print(d.getBalance())
408.04
>>> print(d.getHistory())
[400, 404, 408.04]
```